

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Restaurant Website & Online Ordering System

A Web-Based Platform for Menu Browsing, Table Reservation, Online Food Ordering and Order Tracking

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech – 3rd Semester
Prepared By	Kakul Saini (22515000059)
Project Guide	Mr. Gautam Mukherjee
Date	15/08/2026

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Project Scope.....	1
1.5 References.....	1
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Features (Summary).....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints.....	3
2.6 Assumptions and Dependencies.....	3
3. System Features (Functional Requirements).....	3
3.1 Module: User Registration & Authentication.....	3
3.2 Module: Customer Portal.....	4
3.3 Module: Staff Portal (Waiter / Kitchen).....	4
3.4 Module: Administrator Portal.....	5
3.5 Module: Notifications.....	5
4. External Interface Requirements.....	6
4.1 User Interfaces.....	6
4.2 Hardware Interfaces.....	6
4.3 Software Interfaces.....	6
4.4 Communication Interfaces.....	6
5. Use Case Summary.....	6
5.1 Primary Flow – Order Placement (Customer).....	7
6. Non-Functional Requirements.....	7
6.1 Performance Requirements.....	7
6.2 Security Requirements.....	8
6.3 Usability.....	8
6.4 Reliability & Availability.....	8
6.5 Maintainability & Portability.....	8
6.6 Scalability.....	8
7. Software Testing Considerations.....	9
8. Proposed System Architecture.....	9
8.1 Suggested Technology Stack.....	10
9. Data Model Overview (for Database Planning).....	10
9.1 Core Entities.....	10
9.2 Key Relationships.....	10
10. Future Enhancements (Out of Current Scope).....	11
11. Glossary.....	11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Restaurant Website & Online Ordering System (RWOOS), a web-based application designed to simplify how customers browse the menu, reserve tables and place food orders (dine-in, takeaway or delivery) at a restaurant, and how restaurant staff manage those orders and reservations. The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword “shall” to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short) and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- Student / Development Team – to understand what to build, module by module.
- Project Guide / Faculty Evaluator – to assess completeness, correctness and feasibility of scope.
- UI/UX Designer – to design screens based on Section 4 and Section 3 workflows.
- Database / Backend Developer – to design schema and APIs based on Section 8 and Section 9.
- Software Tester – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

The Restaurant Website & Online Ordering System will allow customers to register, browse the digital menu by category, view item details (price, description, images, dietary tags), place orders for dine-in, takeaway or home delivery, reserve a table for a chosen date and time, and track the live status of their order. Restaurant staff (waiters/kitchen staff) will be able to view and update incoming orders, manage table reservations, and mark orders as prepared/served. Administrators (restaurant owner/manager) will manage the menu, categories, tables, staff accounts, and view operational reports.

The scope is deliberately kept realistic for a college-level team project. The system will manage menu browsing, table reservation and order placement/tracking — it is not a full Restaurant/Point-of-Sale (POS) or accounting system. Online payment gateway integration, third-party delivery-partner integration, loyalty/rewards programs and multi-branch/franchise management are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.

- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Node.js, Express.js and MongoDB official documentation.
- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

RWOOS is a new, self-contained, independent web application. It is not a modification of an existing restaurant system. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8) so that it can later be integrated with external systems such as SMS/payment gateways as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- Customer registration, login and profile management.
- Digital menu browsing with category filters and item search.
- Table reservation with real-time availability display.
- Online order placement for dine-in, takeaway or delivery, with cart management.
- Live order status tracking (Placed → Preparing → Ready/Out for Delivery → Completed).
- Staff dashboard for managing incoming orders and reservations.
- Admin dashboard for managing menu items, categories, tables and staff.
- Automated email notifications for order and reservation confirmations.
- Customer ratings and reviews for menu items.
- Basic analytical reports (daily sales, popular items, table occupancy).

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Customer	General public browsing the menu and placing orders/reservations.	Basic computer/smartphone literacy; primary end-user of the ordering flow.
Waiter / Front-of-House Staff	Assists dine-in customers and updates table/order status.	Trained internal staff; uses the system on a tablet/desktop during shifts.
Kitchen Staff (optional)	Views incoming orders and updates preparation status.	Basic technical familiarity; uses a simplified kitchen-display view.
Administrator / Manager	Restaurant owner or manager – super-user.	Trusted internal user with full configuration and reporting access.

2.4 Operating Environment

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- Server side: Node.js runtime environment hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.

- Database: MongoDB Atlas (cloud) or local MongoDB instance.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep the project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Payment gateway integration and SMS gateway integration are treated as optional/future add-ons due to cost and verification overhead.
- The system assumes a single restaurant / single branch deployment (multi-branch/franchise support is a future enhancement).

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms and email.
- The restaurant's menu, categories and tables are set up by the Administrator before customers begin ordering.
- Reliable internet connectivity is available at both client and server ends.
- Email delivery depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows customers and staff to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new customer to register using name, email, phone number and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users (Customer, Staff, Admin) via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall restrict access to module features based on the logged-in user's role (Customer/Staff/Admin).	High
FR-07	The system shall allow a logged-in user to log out, invalidating the active	Medium

ID	Requirement	Priority
	session.	

3.2 Module: Customer Portal

Description: Core browsing, ordering and reservation experience for customers.

ID	Requirement	Priority
FR-08	The system shall allow a customer to browse the menu by category (starters, main course, desserts, beverages, etc.).	High
FR-09	The system shall display each menu item's details including price, description, image and dietary tag (veg/non-veg/vegan).	High
FR-10	The system shall allow a customer to search menu items by name or keyword.	Medium
FR-11	The system shall allow a customer to add/remove items to a cart and adjust item quantity before checkout.	High
FR-12	The system shall allow a customer to place an order for dine-in, takeaway or delivery, choosing address/table as applicable.	High
FR-13	The system shall allow a customer to reserve a table by selecting an available date, time slot and party size.	High
FR-14	The system shall prevent two customers from reserving the same table for an overlapping date/time slot (no double-booking).	High
FR-15	The system shall allow a customer to view a list of their upcoming and past orders and reservations.	High
FR-16	The system shall allow a customer to cancel an upcoming order or reservation within the allowed time window.	High
FR-17	The system shall allow a customer to track the live status of a placed order (Placed/Preparing/Ready/Out for Delivery/Completed).	Medium
FR-18	The system shall allow a customer to view and update their own profile and saved delivery addresses.	Medium
FR-19	The system shall allow a customer to rate and review a menu item after order completion.	Low

3.3 Module: Staff Portal (Waiter / Kitchen)

Description: Enables restaurant staff to manage incoming orders and table reservations.

ID	Requirement	Priority
FR-20	The system shall display all incoming orders in real time, grouped by status (New/Preparing/Ready/Served).	High
FR-21	The system shall allow staff to update an order's status as it progresses through preparation and service.	High

ID	Requirement	Priority
FR-22	The system shall allow staff to view and manage the day's table reservations, including marking a table as occupied/free.	High
FR-23	The system shall allow staff to record a walk-in dine-in order directly on behalf of a customer.	Medium
FR-24	The system shall allow staff to cancel an order with a reason, triggering a customer notification.	Medium
FR-25	The system shall allow kitchen staff to view a simplified order-ticket view listing items and special instructions only.	Low

3.4 Module: Administrator Portal

Description: System configuration, oversight and reporting.

ID	Requirement	Priority
FR-26	The system shall allow the Admin to add, edit or remove menu items and categories, including price and availability.	High
FR-27	The system shall allow the Admin to manage restaurant tables (add/edit/remove) with seating capacity.	High
FR-28	The system shall allow the Admin to add, edit, deactivate or remove staff accounts.	High
FR-29	The system shall allow the Admin to view and manage all registered customer accounts.	Medium
FR-30	The system shall allow the Admin to view all orders and reservations across the restaurant, with filters by date, status or table.	High
FR-31	The system shall allow the Admin to generate reports on daily/weekly sales, popular menu items and table occupancy.	Medium
FR-32	The system shall allow the Admin to post special offers/announcements visible to customers on the home page.	Low

3.5 Module: Notifications

ID	Requirement	Priority
FR-33	The system shall send an email confirmation to the customer immediately after a successful order or reservation.	High
FR-34	The system shall send an email notification when an order or reservation is cancelled or its status changes.	Medium
FR-35	The system shall send an automated reminder email prior to an upcoming table reservation (e.g. 2 hours before).	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page – restaurant introduction, featured dishes, offers, reservation and order call-to-action.
- Customer Dashboard – browse menu, cart & checkout, reserve table, view/track/cancel orders and reservations, profile.
- Staff Dashboard – live incoming-order board, table/reservation manager, order-status updates.
- Admin Dashboard – manage menu/categories/tables/staff, order & reservation overview, reports and charts.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.4) using a warm, appetite-inviting colour palette suitable for a restaurant brand.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB. A tablet at the host/waiter station is recommended, but not mandatory, for live order management.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component library such as Bootstrap/Tailwind/MUI)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) – NoSQL document store
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications
Hosting (suggested)	Frontend – Vercel/Netlify; Backend – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client–server communication shall occur over HTTPS using RESTful JSON APIs.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.
- (Future phase) SMS notifications may be integrated via a gateway such as Twilio.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (pre-conditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
Customer	Register / Login	Create an account or sign in to access ordering and reservation features.
Customer	Browse Menu	View menu items by category with price and description.
Customer	Reserve Table	Select an available date/time slot and confirm a table reservation.
Customer	Place Order	Add items to cart and confirm a dine-in, takeaway or delivery order.
Customer	Track / Cancel Order or Reservation	View live status and manage existing orders/reservations.
Customer	Rate Menu Item	Leave a rating and review after order completion.
Staff	Manage Incoming Orders	View and update order status through preparation and service.
Staff	Manage Reservations & Tables	See the day's reservations and mark tables occupied/free.
Admin	Manage Menu & Categories	Add/edit/remove menu items, categories and prices.
Admin	Manage Tables & Staff	Add/edit/remove tables and staff accounts.
Admin	View Reports	Analyse sales, popular items and table occupancy.

5.1 Primary Flow – Order Placement (Customer)

1. Customer logs in.
2. Browses menu by category and adds items to the cart.
3. Reviews the cart and selects order type (dine-in/takeaway/delivery).
4. Confirms order details (table, address or takeaway time).
5. Confirms and submits the order.
6. System saves the order and sets its status to 'Placed'.
7. Confirmation email is sent to the customer and the order appears on the staff dashboard.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium

ID	Requirement	Priority
NFR-03	Order and reservation confirmation shall be processed and reflected within 2 seconds of submission.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal or order data shall require a valid authentication token.	High
NFR-06	Role-based authorization shall ensure a Customer cannot access Staff/Admin-only endpoints.	High
NFR-07	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-08	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time customer to place an order in under 5 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Order and reservation data shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance and extension by new student contributors.
- The system shall run on any OS capable of hosting Node.js and MongoDB (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a single restaurant/branch in the academic version, the data model and API design shall avoid hard-coded single-restaurant assumptions where reasonably possible, so that multi-category and, eventually, multi-branch support can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-12	Place an order with items in the cart	Order created; confirmation shown; email sent.
TC-02	FR-14	Attempt to reserve an already-booked table for an overlapping slot	System rejects with a clear 'table unavailable' message.
TC-03	FR-16	Cancel an upcoming reservation	Reservation status changes to 'Cancelled'; table slot re-opens.
TC-04	FR-06	Customer attempts to access Admin-only page	Access denied (403) / redirected to login.
TC-05	FR-02	Register with a weak password	Registration blocked with validation message.

Recommended testing types: Unit testing (backend controllers/validators using a framework such as Jest/Mocha), Integration testing (API endpoints with Postman/Jest+Supertest), and Manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a MERN stack (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended. This keeps the project fully JavaScript-based end-to-end, is well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

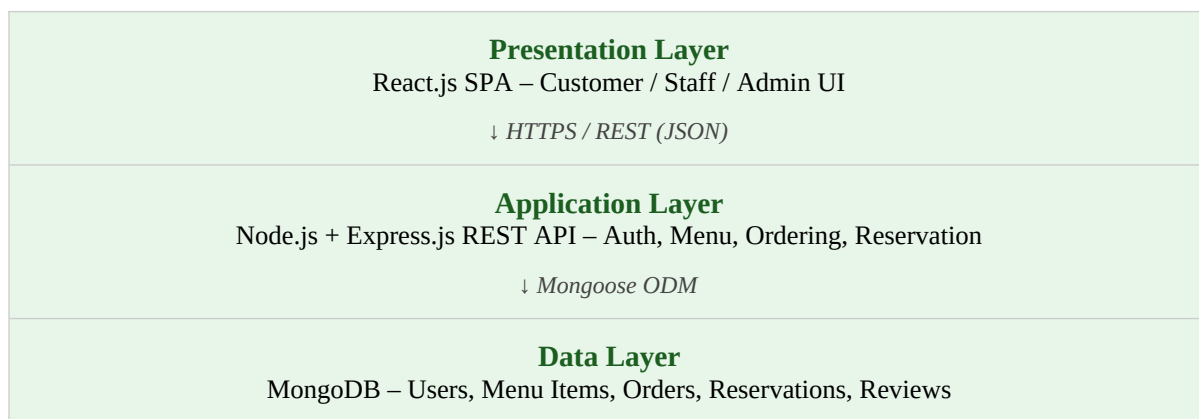


Figure 8.1: Three-tier architecture of the Restaurant Website & Online Ordering System.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt, Nodemailer
Database	MongoDB with Mongoose
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

Entity	Key Attributes
User	_id, name, email, passwordHash, role (customer/staff/admin), phone, createdAt
Customer (extends User)	addresses[], favoriteItems[]
MenuItem	_id, name, description, price, categoryId, imageUrl, isVeg, isAvailable
Category	_id, name, description
Table	_id, tableNumber, capacity, isActive
Reservation	_id, customerId, tableId, date, timeSlot, partySize, status (Pending/Confirmed/Cancelled/Completed)
Order	_id, customerId, items[], orderType (DineIn/Takeaway/Delivery), tableId/address, status (Placed/Preparing/Ready/OutForDelivery/Completed/Cancelled), totalAmount, createdAt
Review	_id, customerId, menuItemId, rating, comment, createdAt
Notification	_id, userId, message, type, isRead, createdAt

9.2 Key Relationships

- A Category has many MenuItems (one-to-many).
- A Customer has many Orders and many Reservations (one-to-many).
- An Order references many MenuItems through its items[] list (many-to-many via Order).
- A Table has many Reservations over time (one-to-many).
- A MenuItem can have many Reviews, each created by one Customer (one-to-many).
- A User can receive many Notifications (one-to-many).

10. Future Enhancements (Out of Current Scope)

- Online payment gateway integration for order checkout.
- SMS notifications alongside email.
- Loyalty points / rewards program for repeat customers.
- Integration with third-party delivery partners.
- Multi-branch / franchise management.
- AI-based menu recommendation based on order history.
- Kitchen Display System (KDS) hardware integration.

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
POS	Point of Sale – a system used to process in-store transactions
UI/UX	User Interface / User Experience

End of Document – Restaurant Website & Online Ordering System SRS v1.0